Applied Nanoscience (2023) 13:1655–1664
https://doi.org/10.1007/s13204-021-02021-0

ORIGINAL ARTICLE

Microsoft Azure active directory for next level authentication to provide a seamless single sign-on experience

D. Subbarao[1] · Bhagya Raju[2] · Farha Anjum[2] · Ch venkateswara Rao[3] · B. Mahender Reddy[3]

Received: 16 June 2021 / Accepted: 8 August 2021 / Published online: 29 September 2021

Abstract

Authentication is crucial although if system which facilitates secure their networks by limiting access to protected resources such as networks, websites, network-based software, databases, and other computer systems or services to only authenticated users (or processes). In general, modern authentication protocols such as Security Assertion Markup Language 2.0 (SAML), WS-Fed, OAuth, and OpenID discourage apps from handling user credentials. The aim is to keep an app's authentication method and its functionality separate. Azure Active Directory (Azure AD) manages the login process to keep confidential data (such as passwords) out of the hands of websites and apps. This allows identity providers (IdP) like Azure AD to provide seamless single sign-on experiences, allow users to authenticate using factors other than passwords (phone, face, biometrics), and block or elevate authentication attempts if Azure AD detects, for example, that the user's account has been compromised or that the user is attempting to access an app from an untrusted location. The main goal of the work is Converting Visual Studio from ADAL to MSAL has allowed us to better support Conditional Access and Multi-factor Authentication and other new AAD features which benefit our customers. Visual Studio 2019 and the.NET Core SDK can be used to complete this work. The SAML request–response authentication workflow between these providers is checked to ensure that user login information is accurate and safe.

Keywords Azure AD · Security Assertion Markup Language 2.0 (SAML) authentication · .NET · Identity management as a service · Single sign-on

Introduction

Nowadays people are moving towards the cloud-based platforms. The adoption rate is gradually increasing, due to a greater number of users accessing the provided resources and services on demand. Despite several advantages, cloud delivery models bring challenges and issues that need to be solved before the adoption of the above paradigms. Surveys conducted by Microsoft, NIST, and Cloud Security Alliance (CSA) show that security is one of the major challenges in cloud computing (Rupa et al. 2020). Thus, many large enterprises, Server Message Block (SMB), startup companies, and the user's adapting to any one of the delivery models as per their business requirements. Listed several security issues related to the service models (i.e., IaaS, PaaS, SaaS) in terms of data isolation, data integrity, authentication, authorization, and web application security, etc.

State of art

In some traditional on-premise applications, the administration of the sensitive data resides in each enterprise boundary with various security controls. It even relies on on-premises infrastructure services, such as AD to provide the necessary identity information. However, with the SaaS model when the user tries to access a cloud application or one of the applications of the on-premises model accessing a cloud service, the same identity usually does not work. Where does it get the identity information? A federated identity management system (IMS) can address these issues by implementing various models of authentication and access control for

✉ D. Subbarao
subbu.dasari@gmail.com

[1] Siddhartha Institute of Engineering and Technology, Ibrahimpatnam, Hyderabad, India

[2] ECE Department, Siddhartha Institute of Engineering and Technology, Ibrahimpatnam, Hyderabad, India

[3] CSE Department, Siddhartha Institute of Engineering and Technology, Ibrahimpatnam, Hyderabad, India

internal and web-based applications (Ferdous et al. 2020). Additionally, management and auditing of user's profiles become simplified with this centralized IS. In today's market, there are many authentication mechanisms available for accessing the internal and cloud (SaaS) applications with the same credentials, they are OpenID Connect, OAuth, and SAML and so on. This paper focuses on developing a SaaS web application with built-in authentication service to the users that can be later applied to hybrid environments. The solution is based on implementing the SAML protocol with SSO feature for identity federation and authenticating users securely. This SAML authentication service enables a smooth but secure and trustworthy communication for user's authentication information. This developed application must be easily uploaded to a cloud provider without any major changes in the application design and data.

Objective

The goal of an IMS is to ensure that only authenticated users have access to specified resources in hybrid environments. In case of malfunction, it raises privacy issues with respect to access compliance and governance. Currently, there are various open source and proprietary IAM solutions available in the market. The selection of the right solution within the enterprise boundaries or cloud depends on various factors, such as scope, feature set, ease of deployment, scalability, security, privacy, and cost. In general, larger organizations opt for distributed IAM solutions for efficiency and effectiveness, as implementation failure results in complex system management issues for user identities. In the case of SMB and startup companies, a single software solution or identities managed through a single implementation tool can be applied across the business systems to fully support end-users within an organization (Verzeletti et al. 2018a). Further such implementation will also benefit large organizations in terms of reduction in application cost, management overhead, and security. This work focuses on how the developed browser-based SaaS web application provides SAML authentication for user's on-premises and cloud.

- The app gets a token from the user's username and password, then uses the Microsoft Graph to get details about the user and their manager.
- Although Username/Password is required in certain situations (for example, DevOps scenarios), it is not recommended because:
- It requires having credentials in the application, which is not the case with the other flows;
- It should only be used when the resource owner and the client have a high level of trust and when other authorization grant types are not available (for example, an authorization grant).
- It's worth noting that with Azure AD-registered applications, attempts to authenticate and obtain tokens for users using this flow will often fail. The following are some of the conditions and circumstances that will result in failure.
- When the user must agree to the application's requests for permissions.
- When a multi-factor authentication conditional access policy is in effect.
- If this user account is compromised, Azure AD Identity Protection will block authentication attempts.
- The user's password has run out and needs to be reset.

Although this flow appears to be simpler than others, it often causes more issues in applications than other flows such as authorization code grant. The error handling is also quite complex (detailed in the sample) (Rupa et al. 2020). Developers who wish to gain good familiarity of programming for Microsoft Graph are advised to go through the "An introduction to Microsoft Graph for developers" recorded session.

Literature review

To improve security, Indu et al. (2017) proposed an integrated IMS for cloud web services. They put the proposed system into action by integrating SAML technology with a token-based authentication system for fine-grained authentication. Metadata files are used to create contact between the IDPs, service providers, and site providers. The SAML attributes (or certificates) in these metadata files are used to protect authentication between the source and destination (Grabatin and Hommel 2018). The IDP produces access tokens after the user successfully authenticates to cloud web services using SSO SAML authentication. These copies of access tokens are sequentially stored within the IDP and service provider. These access tokens are withdrawn from the service and IDP's token store after they have been used for any web service request from cloud servers. This prevents the same token from being used again. They provided a hashing algorithm that the IDP could use to generate these access tokens in a hexadecimal random string. For the generation of the access token, the string considers parameters such as the user's name, the service provider entity id, the current time, and the token expiry time. They used SAML protocol metadata files to enforce this token-based authentication mechanism. They worked with the cloud service provider to modify the current IdP and SP metadata files to validate the access token and react to the service providers, respectively. They presented a performance assessment based on time complexity (the total time it takes to execute the token generation algorithm), a comparison of

the proposed SAML-based authentication model to current authentication models, and a security review of the adapted SAML protocol in terms of insider and outsider cyber-attacks.

Eludiora et al. (2011) achieve implementation of Web Single Sign-On (SSO) authentication using the standard framework SAML 2.0. To implement the standard, they have used an opensource Simple SAML php library. Their library is written in PHP and offers support for integrated web-based PHP application into a federation. The mechanism of SSO permits user's authentication and authorization access to computers/application resources only once, without the need for multiple passwords. Whereas, the Web SSO features using the SAML allows users to authenticate once between multiple web applications using established trust relationship, not necessarily within the same security domain (Basney et al. 2020). Their experimental setup includes the functionality of two federation scenarios: SAML as a Service Provider and SAML as an IDP are two different things. To authenticate users, these Simple SAML php as a service providers bind to the IdP. Simple SAML php using API classes are included in these implementations. Check if the user is authenticated, need authentication, login, logout, get user attributes, and get login and logout URLs are all included in the API. Thus, the web application has no knowledge of the IdP and allows no change in the web application code. This Simple SAML php as an IDP can accept connections from various service providers (SP) using trusted certificates. On receiving the user request from any of the SP, it validates the credentials against various authentication modules, such as LDAP, CAS, SQL, OpenID, Facebook, and Twitter. After login, the session information is stored by the IdP. This experimental setup is carried on two local computers, one as an SP, and other as an IdP. On the SP side, a sample page was used for authentication. For configuration, the required application modules (Apache web server, PHP) are loaded for SP to connect to the IdP, additionally includes metadata for login/logout URLs and IdP certificate. On the IdP side, they used a SQL authentication module and stores the credentials in a table format. For configuration, includes the authentication method and metadata of SP. The test was successful, once the user access one of the applications, he/she directly goes to the second application using the same IP. These methods only work within the same browser window.

Michael and Anna suggest an identity federation solution that improves authentication, authorization, and accounting (AAA). They list a few basic vendor product improvements that can dramatically boost application-level AAA in mutual Assertion Consumer Service environments (ACS). The ACS is the main component used for implementing SAML standard for service providers. (2019) On SP-side, changes in the SAML metadata includes, ACS acts as a SAML protocol proxy instead of application SAML actual entity names. This information about the application is conveyed to IdP in the request form. The IdP responses to the application entity without knowing on whose behalf those federations requests are made. The author lacks in providing a detail explanation of how application-level granularity is achieved using different vendor products. But a clear idea is given on how SAML standard 2.0 can be used for web SSO authentication and importance of ACS.

Tanimoto et al. present some of the security challenges that encounter while integrating traditional enterprise infrastructure with cloud computing. They focused mainly on user repositories, authentication, and authorization systems within the enterprise security boundary and their adoption to cloud offerings (2019). The author referred to cloud offerings as XaaS (w.r.t IaaS, PaaS, SaaS). They analyzed these integration issues are crucial and need to take an extra care when considering cloud offerings for enterprise IT. This analysis presents a few integration challenges, red-flag pitfalls and identifies best practice/mitigation approaches for application owners, architects, service owners, and cloud providers.

Armando et al. identify many authentication vulnerabilities in emerging SSO protocols such as SAML SSO and OpenID. They discovered a cross-site scripting attack in Google Apps SAML-based SSO and Novell Access Manager V.3.1 SSO (2013). Apart from the demonstration and mitigating the attack, they also described general solutions for the flaws in both protocol implementation. This project study focuses on protocol SAML SSO. Hence, discussion about OpenID is out of scope. If readers are interested, can go through the complete article.

Background

IDaaS (Identity Management as a Service) is a modern cloud computing model that allows companies and organizations to outsource one of the most basic and essential services. Because of its importance in the processes of access control, authorization, and accountability, it is one of the most widely used resources within businesses and organizations. The key drivers for this externalization are cost savings and the elimination of time-consuming activities associated with handling identity services (Chagas et al. 2019). Cloud computing, on the other hand, has sparked widespread debate about the inversion of data power. IDaaS allows companies and organizations to outsource identity management systems from their internal infrastructures and host them on a cloud provider, allowing them to move from on-premise to on-demand identity management. IDaaS also provides cloud providers and suppliers with a new revenue stream, enabling them to extend their service offerings. IDaaS, on the other

hand, offers a variant of one of the traditional cloud computing issues: a lack of control over outsourced data, in this case, information about users' identities.

In this proposed project, a prototype will be developed and demonstrated to show how an IDaaS system that protects users' data privacy can be implemented. The SAML (Security Assertion Markup Language) identity management protocol (Verzeletti et al. 2018b) and a proxy re-encryption scheme will be used to ensure cryptographic protection for this prototype. This approach protects users' privacy by allowing an identity provider to serve attributes to other parties without being able to read their values. It will be compared to other Identity Management Protocols to see how effective it is.

Why SAML & proxy re-encryption schemes?

• SAML is a standardized, safe, and user-friendly protocol.

• SAML is used to establish a single point of authentication at a protected identity provider, which means that user credentials never leave the firewall, and SAML is then used to claim the identity to others.

• This eliminates the need for applications to store or synchronize identities, resulting in fewer points where identities can be compromised or stolen.

SAML also adds a layer of protection using Public Key Infrastructure (PKI) to secure claimed identities from attack attempts.

• For added confidentiality, the Proxy Encryption Scheme is used.

MSAL.NET (Microsoft.Identity.Client) is an authentication library that lets you get tokens from Azure AD and use them to access secure Web APIs (Microsoft APIs or applications registered with Azure Active Directory). MSAL.NET is available on a number of different .NET platforms (Desktop, Universal Windows Platform, Xamarin Android, Xamarin iOS, Windows 8.1, and.NET Core).

Methods

SMAL

This section describes SAML (Security Assertion Markup Language) used in this study. It is an open standard protocol developed by Security Services Technical Committee of "Organization for the Advancement of Structured Information Standards (OASIS)", for establishing secured communication of identity information between participating entities (Indu et al. 2017). The entities involved are generally referred to as, the Identity provider (IdP) and Service provider (SP). The role of the IdP entity is to verify and validate the identity of a user asking for a service. In SAML context, they are also known as SAML authorities and 'Asserting' parties. Whereas, the role of SP entity is to use the IdP to validate the identity and provide the requested service to the user. In SAML context, they are also known as 'Relying' parties because they rely on information provided by asserting party.

It is an XML-based protocol framework, platform-independent, and working specification combines six main components:

- Security assertions,
- Protocols,
- Bindings,
- Profiles,
- Metadata and,
- Authentication context

Security assertions

The exchanging of identity information in the form of security assertions. These assertions with a pre-defined syntax are set of statements about an entity. There are three sets of statements that hold exchange information, such as user authentication, entitlements, and attribute (Bhat 2015). The user authentication statements indicated whether the user is authenticated or not. If the authentication is successful, it must provide the details of the authentication method used and time of authentication from relying upon parties (SP) to (IdP). The entitlement statements indicated whether the authenticated user is subjected to perform the desired action. These action decisions are provided through access control rights. (Bradford et al. 2014) The attribute statement indicates whether the defined attribute for example name, age, job and so on, are used in access control decision making between relying parties.

Protocols

The protocols are used for issuing and exchanging the above security assertions in a form of the package between the SP and IdP. Such protocols are authentication request, request–response, assertion query, artifact resolution and so on. Each package specification varies with a set of rules and SAML elements such as, request and response elements that the IdP or SP should follow.

Bindings

The SAML binding is a mapping of the above protocol messages with standard messaging formats and/or communication protocols. Some of these bindings are SAML SOAP, HTTP Redirect(GET), HTTP Post, SAML URI, and so on.

Profiles

The SAML profile is a technical description of how SAML assertions, protocols, and bindings combine to address a specific use case of a project or a deployment scenario. Some of these profiles are Web browser SSO, Single logout profile, Enhanced client or proxy (ECP), IdP discover profile, SAML attribute profile, and so on. The most important and extensively used profile is the Web Browser SSO profile. For example, if the Web SSO Profile is used for SAML authentication, it details how authentication assertions are exchanged between entities (including what SAML protocols and bindings are used).

Metadata

The SAML metadata defines a configuration data in an XML schema format and exchanges this data between the entities. It defines data like what service is available, addresses, operational roles (IdP or SP), bindings, certificates with Key information for encryption and signing and so on. The SP uses the metadata to know how to communicate with the IdP and vice versa.

Authentication context

The service provider request IdP to uses some type of authentication context for the user. This information includes the type and strength of authentication. The relationship between these components is like building blocks as shown in Fig. 1. When they are put together, provides flexibility and extensibility to implement a system and supports various business use cases. Among the supported use cases, Web Single Sign-On profile use case and identity federation use case are considered for this study.

Web SSO federated identity

This section describes Web Single Sign-On and federated identity. With the involvement of cloud usage for enterprise applications arises many problems. Among them, user identity management is one of the tops most concern. How such multiple user login identities for a single user can be maintained in both on-premises and cloud accounts, password recovery, etc. Also, an increase in fraud rate and user's negligence in writing down multiple passwords/forgetting possessing an additional burden. To solve these problems, two new solutions took a way through to benefit both the businesses and end-users. They are a single sign-on and new identity management structure call as federated identity management.

Web Single Sign-On (SSO)

Single sign-on (SSO) (Catuogno and Galdi 2014) is one of the best solutions for accessing multiple applications with a single user account. In simple terms, it is an authentication service that allows users to use one set of login credentials (i.e., username and password) to access the multiple application that he/she has rights within a single established

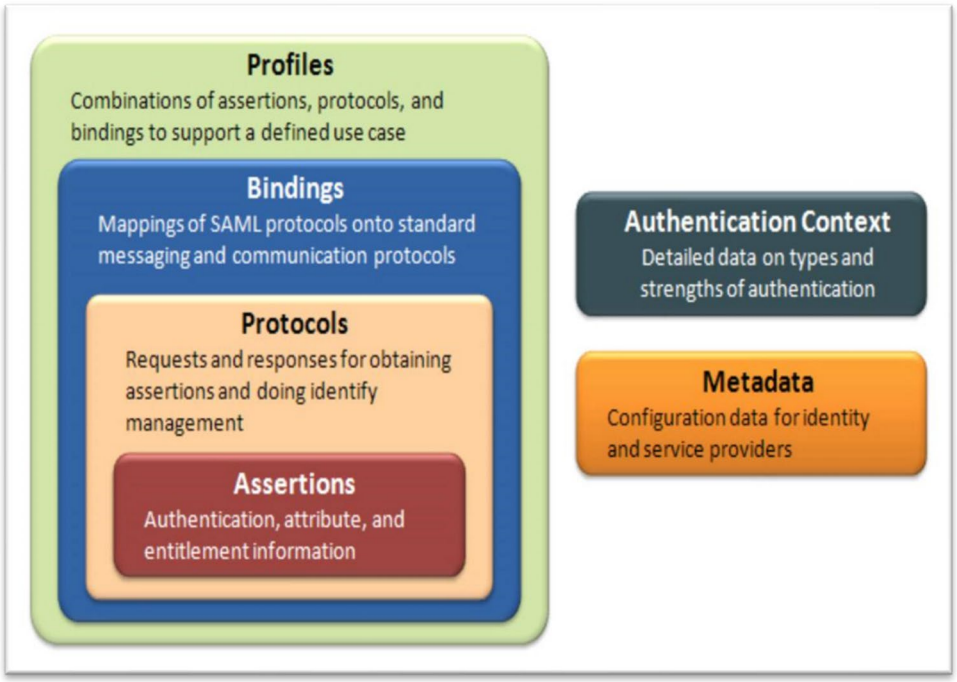

Fig. 1 SAML protocol structure

session. Thus, eliminating the need to switch to multiple sign-on dialogues prompts for login into multiple applications within the active session. When applied to enterprise level, SSO provides a path for business employees to use the same credentials to access public facing sites without the burden of additional login passwords. It eliminates the burden of managing user accounts, cost involved for strong authentication systems and reducing SaaS licensing cost by 30%. There are many SSO solutions available for the business. The implementation solution can be chosen based on business requirements. Considering SAML is used for message communication in this project, then web-browser SSO profile implementation is best suited. Web browser SSO profile uses SAML assertion message, Authentication request protocol, along with bindings HTTP redirects and HTTP POST between the participating entities (i.e., SP and IdP).

Federated identity

The concept of federated identity is to establish an identity trust between a group of organizations/participating entities (Shehu et al. 2019). The goal is to enable secure trust relationships between a groups of organizations/SPs to share information about the identity of its user's (Eludiora et al. 2011). The identity data are stored at one location, i.e., IdP and multiple service providers can enable user authentication to access the required resources. In this way, organizations avoid the burden of managing user's identities. In this study, we are going to use a federated identity management system WSO2 (IdP) for managing user identities. The federation of identities between the SP and IdP can be accomplished using many available open standards, such as SAML, OpenID, OAuth, and so on.

SAML message exchange

This section describes the SP initiated SAML message exchange it fallows the SAML communication algorithm. In the Service Provider (SP) initiated the process, the user accesses the webpage without passing to the IdP first. The service provider will take the responsibility to issue the SAML request back to IdP on behalf of the user. The detail message exchange steps are presented in the below Fig. 2 an explanation as follows:

- The user arrives at the webpage of the company, but without SAML assertion.
- The SP redirects the request to the appropriate IdP login webpage. This is done with the help of exchanging metadata files with attributes data, such as entity ID.
- The SP returns the IdP SSO page to the user browser.
- The user enters the login information. Then the SP initiates the actual SAML authentication process by sending the SAML AuthRequest to the IdP.
- The IdP identifies the user by verifying the details of AuthRequest assertions of SP and its internal user store.
- The IdP provides a SAML response to the SP.
- Based on the response, the SP provides returns the requested resources or an HTTP error.

Algorithm for SAML communication

Algorithm
1: Start
2: **User agent**
3: Get the method from Web browser
4: Return to login page
5: User clicks on "ERL by SAML"
6: SP determines the IdP to use calls from IdP login page () method
7: returns the login page to initiate SAML exchange page
8: User enters the login details
9: Calls the SAML Auth request () method issued by SP
10: IdP Verifies the SP details
11: SAML response () user profile
12: Based on SAML Auth response assertions SP determines to return resource
13: Return User profile ()
14: user http action to logout the application
15: SP calls for application and IdP logout ()
16: subsequent logout messages
17: return to logout page
18: stop

MSAL.NET supports multiple application architectures. It supports all the possible application topologies including native client (mobile/desktop applications) calling the Microsoft Graph in the name of the user, daemons/services or web clients (Web Apps/ Web APIs) calling the Microsoft Graph in the name of a user, or without a user. Apart from these user-agent based client which is only supported in JavaScript. For details about the supported scenarios see Scenarios.

MSAL.NET supports multiple platforms

- .NET Framework,
- .NET Core,
- Xamarin Android,
- Xamarin iOS,
- UWP
- Important

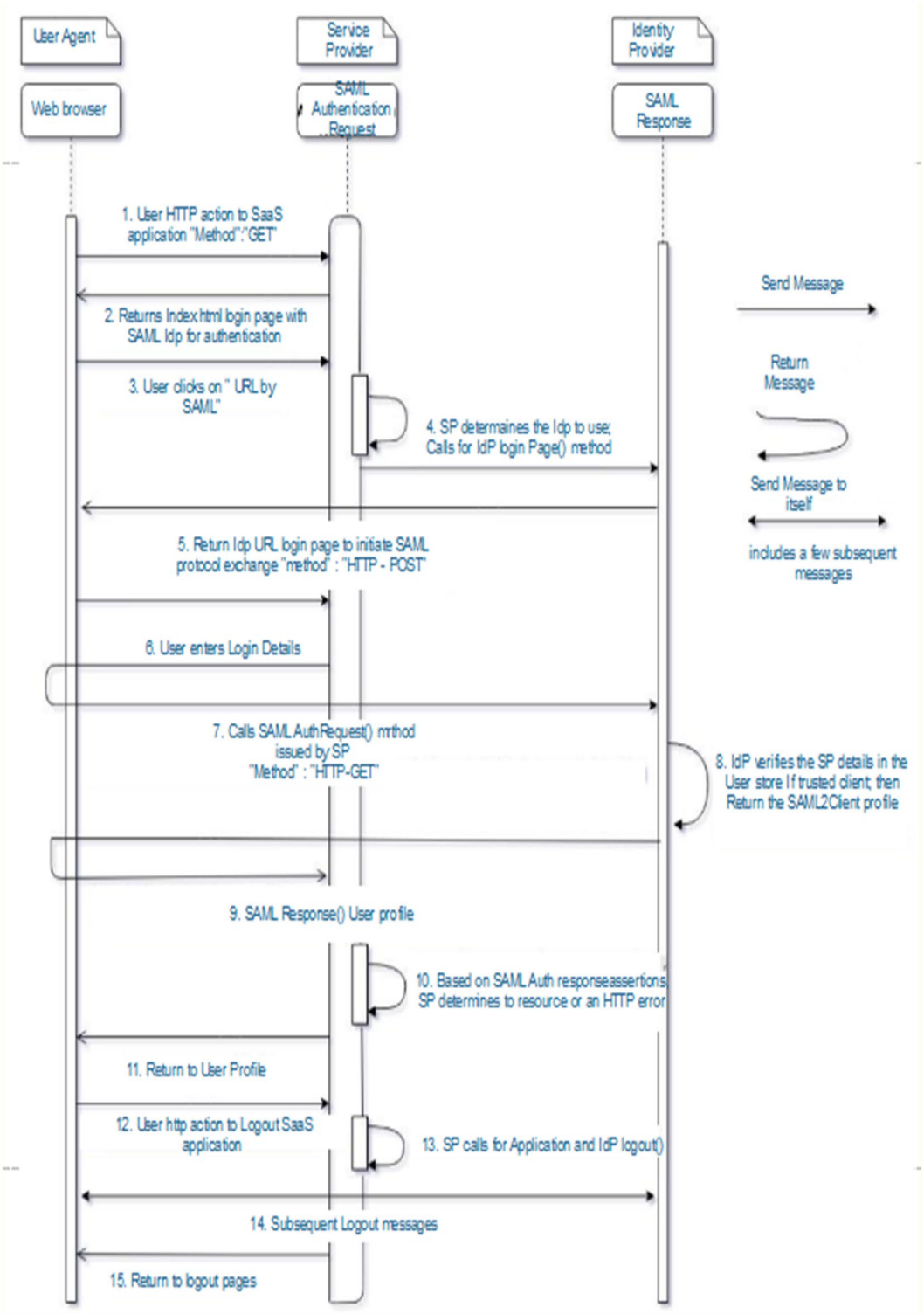

Fig. 2 Sequence diagram of SAML communication between user browser, SP, and IdP (Chagas et al. 2019)

Not all the authentication features are available in all platforms, mostly because:

- they would not make sense in those platforms (for instance iOS and Android applications do not support confidential client flows as these platforms cannot guarantee that application secrets would be safe),

Or because of limitations of the platform itself (for instance .NET Core does not provide UI, and, therefore, acquisition of tokens requiring user interaction through a Web browser is not possible in .NET Core).

Implementations and results

The current section describes the experiment setup, data collection, and methodology. We start with initial design; a demo web application is developed in .net play framework using PAC4J security library for providing SAML authentication between SP and IdP. The web app consists of an index page with an URL to SAML authentication. Here the web app deployed system acts as a service provider. When the user clicks on the SAML URL, then it is redirected to the requested identity server (Wso2) login page. Once the user enters his/her credentials to requests access to a specific resource or service, the identity server validates the authentication request. If the authentication is successful, then it returns a SAML Auth cookie information on the browser along with the profile information.

The redirection from the web app to the WSO2 identity server is established using the X509 certificate. The communication between the SP and IdP is verified by this trusted certificate. For this sample project, to test we have created a self-signed certificate in X509 certificate format. The web app X509 certificate additionally helps in establishing a trusted authentication to the Identity Server. The generated certificate is loaded into the WSO2 identity server trusted Key Store. The role of IS with respect to certificate authentication is; it will authenticate the client (SP) using the client's public key certificate.

After the user enters the credentials, the authentication validation takes place through the exchange of SAML metadata. The developed web application includes a service provider metadata in form of the XML format. The identity provider metadata is downloaded or gets automatically updated to the SP web app. Thus, the identity server verifies the SP metadata, before authenticating the user. A series of SAML communication messages takes place between SP and IP (Fig. 3).

This application has implemented in.NET Core SDK using Visual Studio 2019 to generate the token by calling Microsoft Graph API as depicted in below.

Prerequisites

- Visual Studio 2019 and the.NET Core SDK
- An Internet connection
- A Windows machine (necessary if you want to run the app on Windows)
- An OS X machine (necessary if you want to run the app on Mac)
- A Linux machine (necessary if you want to run the app on Linux)
- A tenant in Azure Active Directory (Azure AD). See how to get an Azure AD tenant for more details about how to get an Azure AD tenant.
- In your Azure AD tenant, a user account. With a Microsoft account, this sample will not work (formerly Windows Live account). As a result, if you used a Microsoft

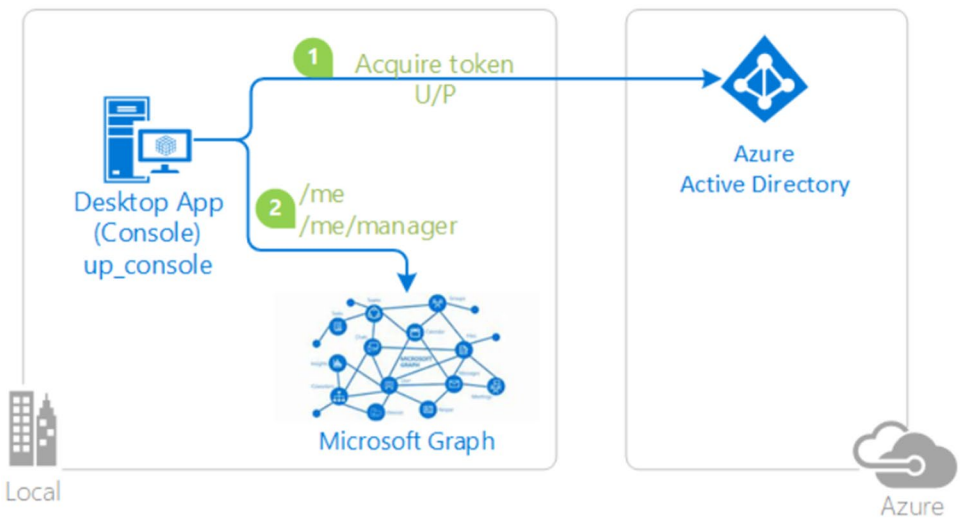

Fig. 3 Architecture diagram for proposed system

account to access the Azure portal and have never established a user account in your directory, you must do so now.

Configure the client project

Open the project in your IDE (like Visual Studio) to configure the code. In the steps below, "ClientID" is the same as "Application ID" or "AppId".

1. Go to the up-console / appsettings.json file and open it.
2. Locate the app key ClientId and substitute the current value with the up-console application's application ID (ClientId) copied from the Azure portal.

How to use PowerSell Script?

Creates Azure AD applications and related objects (passwords, permissions, and dependencies) automatically. This is only applicable to Visual Studio. Change the configuration files for Visual Studio projects, If want to use this automation, expand this section. The permissions are correctly allocated at this stage, but since the client app does not allow users to communicate, they cannot agree to these permissions. To avoid this problem, we would allow the tenant administrator to agree on behalf of all tenants' users. When prompted, press the Grant admin consent for tenant button and select "Yes" to grant consent for the requested permissions for all accounts in the tenant. You must be a tenant administrator to perform this task (Fig. 4).

The output is evaluated using the Technical Risk and Efficacy Evaluation strategy, which was found to be the most appropriate strategy for this paper. This strategy begins with a formative evaluation to determine whether or not the specific technology will function as intended at the outset of the design process. The process then moves on to artificial evaluation, which includes a laboratory experiment to clarify the boundaries of the technology, as well as the cost of setting it up.

Conclusion

We have seen a lot of successful ADAL to MSAL migrations from a wide variety of partner teams. We have migration documentation and will assist with the migration as much as possible, if needed. Here are some testimonials from our happy partner teams! The Azure Portal had a mighty task of migrating from ADAL to MSAL with the constraint of maintaining the current Auth architecture. The MSAL team followed a very systematic migration process. They understood the Azure Portal's Auth architecture, recommended solutions that fit in the current architecture. Following those guidelines, the Portal team was able to build a successful prototype, both teams did a final design review and eventually the changes were formalized for production. All questions, issues, and blockers that came along the way were dealt with in a timely manner and with great patience and communication. Eventually the Azure Portal was able to successfully migrate from ADAL to MSAL without causing any outages in production. Converting Visual Studio from

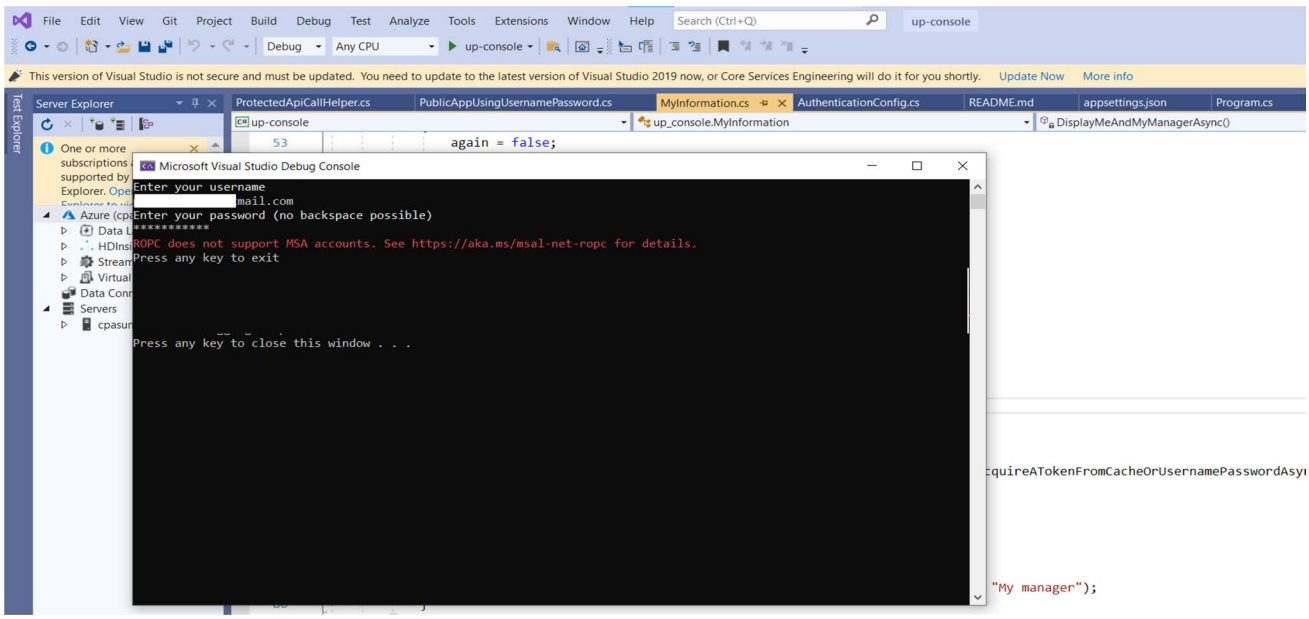

Fig. 4 Output file for the proposed architecture

ADAL to MSAL has allowed us to better support conditional access and multi-factor authentication and other new AAD features which benefit our customers.

In particular, the findings of this research can be beneficial to startups in the early stages of developing user authentication methods and in the search for a single application that can run both on-premises and in the cloud, among other things. According to the findings of this research, the implementation of the authorization module for user authentication can be considered as a possible future work.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

References

Armando A et al (2013) An authentication flaw in browser-based single sign-on protocols: impact and remediations. Comput Secur 33:41–58

Basney J, Cao P, Fleury T (2020) Investigating root causes of authentication failures using a SAML and OIDC observatory. 2020 IEEE 6th International Conference on Dependability in Sensor, Cloud and Big Data Systems and Application (DependSys), Nadi, Fiji: 119–126. https://doi.org/10.1109/DependSys51298.2020.00026

Bhat M (2015) Simulation study of different authentication protocols used for federated identity management in cloud. Int J Emerg Res Manage Technol 4:2

Bradford M, Earp JB, Grabski S (2014) Centralized end-to-end identity and access management and ERP systems: a multi-case analysis using the Technology Organization Environment framework. Int J Account Inf Syst 15:149–165

Catuogno L, Galdi C (2014) Achieving interoperability between federated identity management systems: a case of study. J High Speed Netw 20(4):209–221

Chagas M, Silva JJ, Adriano DD, Wangham MS (2019) SM4VO: a security management mechanism for virtual organizations. 2019 9th Latin-American Symposium on Dependable Computing (LADC), Natal, Brazil: 1–10. https://doi.org/10.1109/LADC48089.2019.8995732

Eludiora S et al (2011) A user identity management protocol for cloud computing paradigm. IJCNS 4:152–163

Ferdous MS, Chowdhury F, Alassafi MO, Alshdadi AA, Chang V (2020) Social anchor: privacy-friendly attribute aggregation from social networks. IEEE Access 8:61844–61871. https://doi.org/10.1109/ACCESS.2020.2981553

Grabatin M, Hommel W (2018) Reliability and scalability improvements to identity federations by managing SAML metadata with distributed ledger technology. NOMS 2018–2018 IEEE/IFIP Network Operations and Management Symposium, Taipei, Taiwan: 1–6. https://doi.org/10.1109/NOMS.2018.8406310.

Indu I, Anand PMR, Bhaskar V (2017) Encrypted token based authentication with adapted SAML technology for cloud web services. J Netw Comput Appl 99:131145

Michael S, Anna ZJ (2019) An Identity Provider as a Service platform for the eduGAIN research and education community. 2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), Arlington, VA, USA: 739–740

Rupa C, Patan R, Al-Turjman F, Mostarda L (2020) Enhancing the access privacy of IDaaS system using SAML protocol in fog computing. IEEE Access 8:168793–168801. https://doi.org/10.1109/ACCESS.2020.3022957

Shehu A, Pinto A, Correia ME (2019) Privacy preservation and mandate representation in identity management systems. 2019 14th Iberian Conference on Information Systems and Technologies (CISTI), Coimbra, Portugal: 1–6. https://doi.org/10.23919/CISTI.2019.8760690.

Tanimoto S, Toriyama S, Iwashita M, Endo T, Chertchom P (2009) Secure operation of biometric authentication based on User's viewpoint. 2019 IEEE International Conference on Big Data, Cloud Computing, Data Science & Engineering (BCD), Honolulu, HI, USA: 166–171. https://doi.org/10.1109/BCD.2019.8885177

Verzeletti GM, de Mello ER, Wangham M (2018a) A mobile identity management system to enhance the Brazilian electronic government. IEEE Latin Am Trans 16(11):2790–2797. https://doi.org/10.1109/TLA.2018.8795121

Verzeletti GM, de Mello ER, Wangham MS (2018b) A National Mobile Identity Management Strategy for Electronic Government Services. 2018 17th IEEE International Conference on Trust, Security and Privacy In Computing and Communications/ 12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE), New York, NY, USA: 668–673. https://doi.org/10.1109/TrustCom/BigDataSE.2018.00098